



CE 222 – Computer Organization and Assembly Language (COAL)

Lecture 28

Dr. Asad Mahmood

April 08, 2026

Lecture Outline

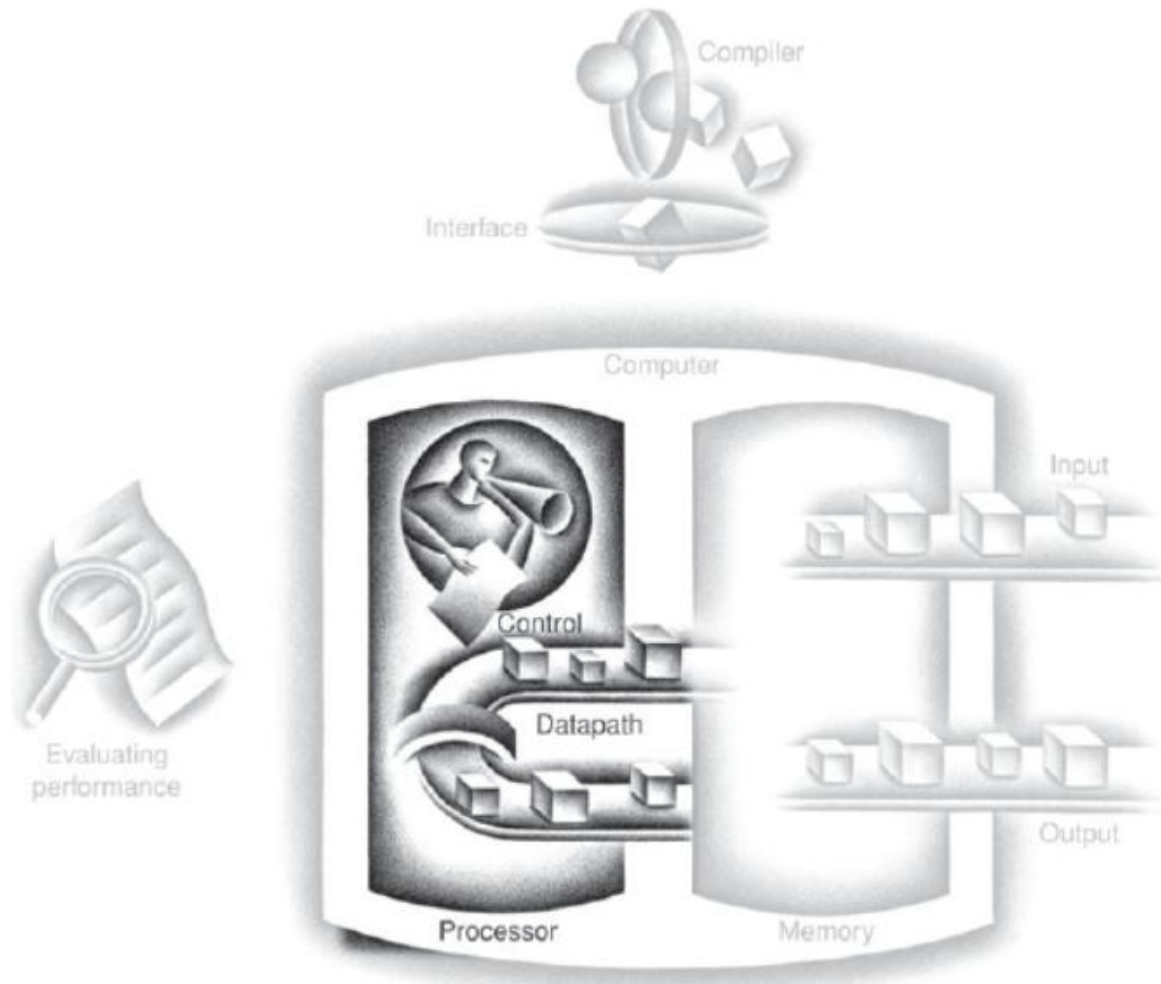
- Announcements (Mid marks, Assignment and Quiz 3)
- Outline of Previous Lecture:
 - **Chapter 4 – The Processor**
 - 4.4 A Simple Implementation Scheme
- Outline of Today's Lecture:
 - **Chapter 4 – The Processor**
 - 4.5 Multicycle Implementation

Chapter 4

The Processor

Chapter Theme

The Five Classic Components of a Computer



Chapter Outline

Chapter 4 – The Processor

- 4.1 Introduction
- 4.2 Logic Design Conventions
- 4.3 Building a Datapath
- 4.4 A Simple Implementation Scheme
- 4.5 Multicycle Implementation
- 4.6 An Overview of Pipelining
- 4.7 Pipelined Datapath and Control
- 4.8 Data Hazards: Forwarding versus Stalling
- 4.9 Control Hazards
- 4.10 onwards (self-reading)

ALU Control

- Let us make a simple implementation of our **RISC-V subset** by **adding control** to the **datapath** we made in the last section.
- We will start by developing **control for ALU**
- ALU used in **arithmetic and logical instructions**
- Role of ALU in **memory-access and branch instruction**.

ALU control	Function
0000	AND
0001	OR
0010	add
0110	subtract

ALU Control

- What can be used to define 'ALU control' bits?
- Assume 2-bit **ALUOp** derived from opcode
 - Combinational logic derives ALU control

Instruction opcode	ALUOp	Operation	Funct7 field	Funct3 field	Desired ALU action	ALU control input
R-type	10	add	0000000	000	add	0010
R-type	10	sub	0100000	000	subtract	0110
R-type	10	and	0000000	111	AND	0000
R-type	10	or	0000000	110	OR	0001
lw	00	load word	XXXXXXXX	XXX	add	0010
sw	00	store word	XXXXXXXX	XXX	add	0010
beq	01	branch if equal	XXXXXXXX	XXX	subtract	0110

ALU Control

- Recalling instruction formats, as studied in chap. 2

Name (Bit position)	Fields					
	31:25	24:20	19:15	14:12	11:7	6:0
(a) R-type	funct7	rs2	rs1	funct3	rd	opcode
(b) I-type	immediate[11:0]		rs1	funct3	rd	opcode
(c) S-type	immed[11:5]	rs2	rs1	funct3	immed[4:0]	opcode
(d) SB-type	immed[12,10:5]	rs2	rs1	funct3	immed[4:1,11]	opcode

ALU Control

- Design of control unit can be done using a truth table
- Don't care terms

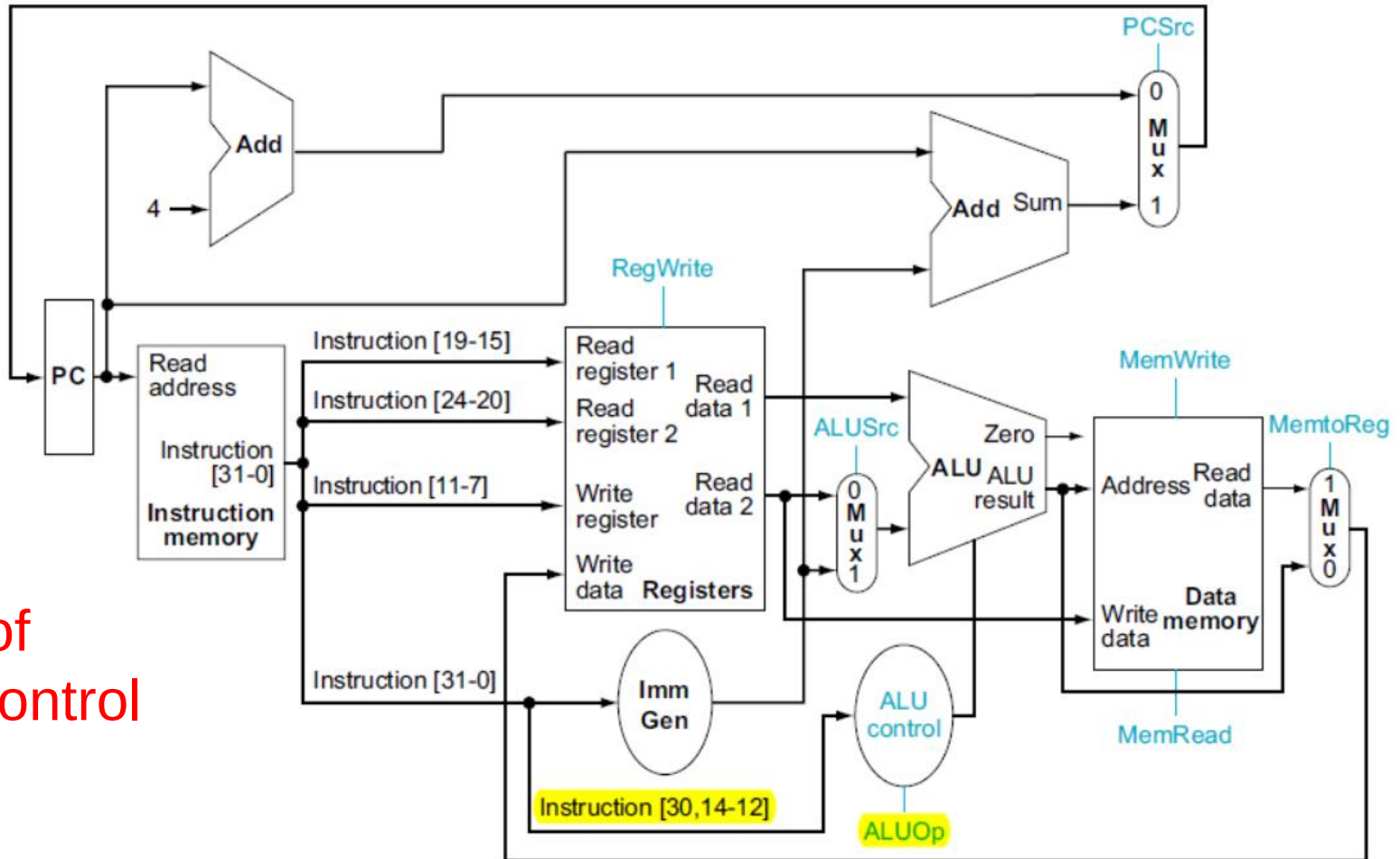
ALUOp		Funct7 field							Funct3 field			Operation
ALUOp1	ALUOp0	I[31]	I[30]	I[29]	I[28]	I[27]	I[26]	I[25]	I[14]	I[13]	I[12]	
0	0	X	X	X	X	X	X	X	X	X	X	0010
X	1	X	X	X	X	X	X	X	X	X	X	0110
1	X	0	0	0	0	0	0	0	0	0	0	0010
1	X	0	1	0	0	0	0	0	0	0	0	0110
1	X	0	0	0	0	0	0	0	1	1	1	0000
1	X	0	0	0	0	0	0	0	1	1	0	0001

The Main Control Unit

- Let us now look at the design of the complete control unit of our simplified processor
- Important to see our target instruction formats and from where to get inputs for the control unit e.g. opcode
- Via Regularity, RISC-V instruction formats help simplify hardware design, as compared to other architectures e.g. MIPS

The Main Control Unit

- Datapath with all control signals



Function of
different control
signals

The Main Control Unit

- Function of different control signals in our simplified processor

Signal name	Effect when deasserted	Effect when asserted
RegWrite	None.	The register on the Write register input is written with the value on the Write data input.
ALUSrc	The second ALU operand comes from the second register file output (Read data 2).	The second ALU operand is the sign-extended, 12 bits of the instruction.
PCSrc	The PC is replaced by the output of the adder that computes the value of $PC + 4$.	The PC is replaced by the output of the adder that computes the branch target.
MemRead	None.	Data memory contents designated by the address input are put on the Read data output.
MemWrite	None.	Data memory contents designated by the address input are replaced by the value on the Write data input.
MemtoReg	The value fed to the register Write data input comes from the ALU.	The value fed to the register Write data input comes from the data memory.

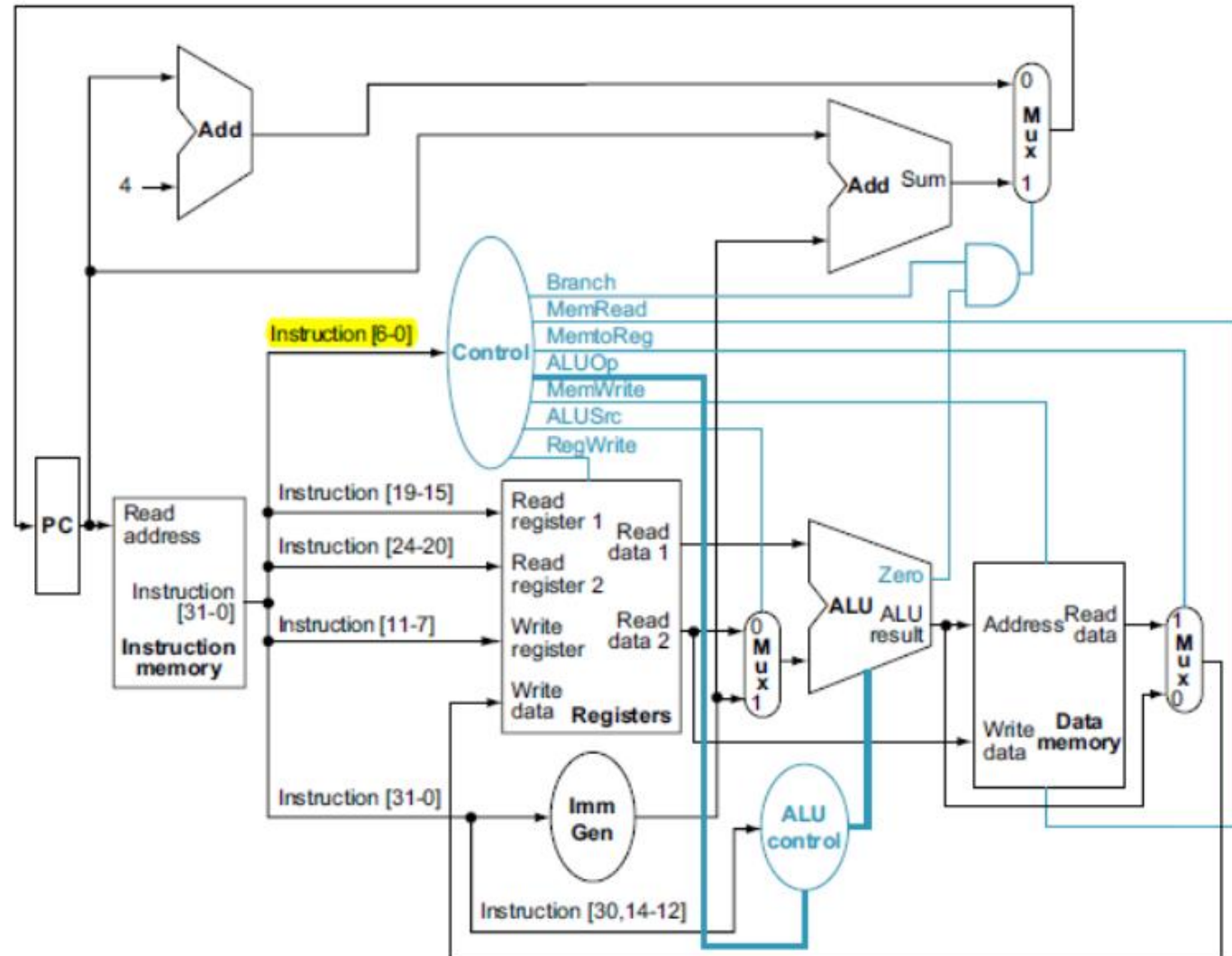
The Main Control Unit

- Values of control signals for different instruction types

Instruction	ALUSrc	Memto-Reg	Reg-Write	Mem-Read	Mem-Write	Branch	ALUOp1	ALUOp0
R-format	0	0	1	0	0	0	1	0
lw	1	1	1	1	0	0	0	0
sw	1	X	0	0	1	0	0	0
beq	0	X	0	0	0	1	0	1

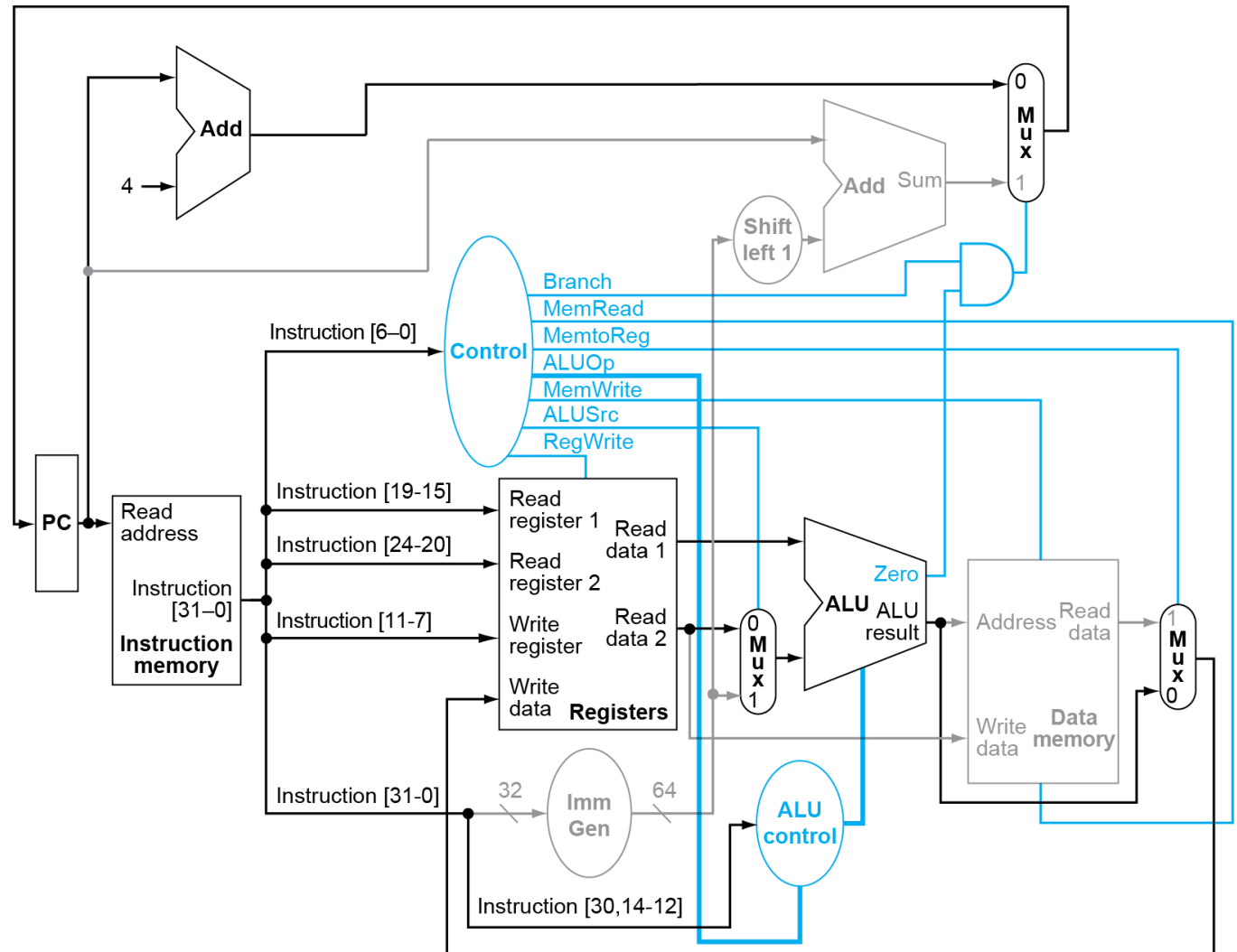
Datapath With Control

The Control Unit sets all the control signals except one!



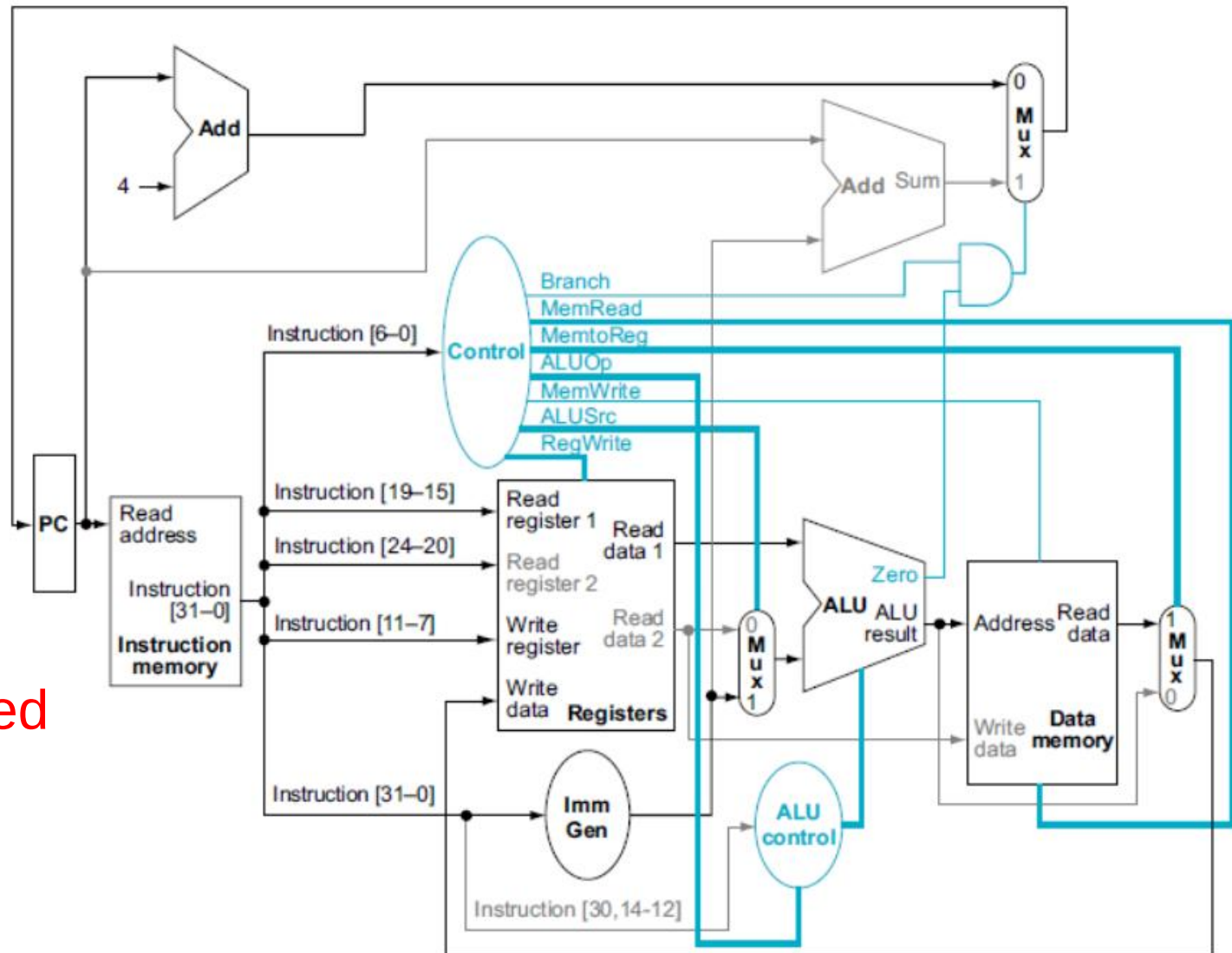
R-Type Instruction

- Instruction Execution steps
- Clock cycles

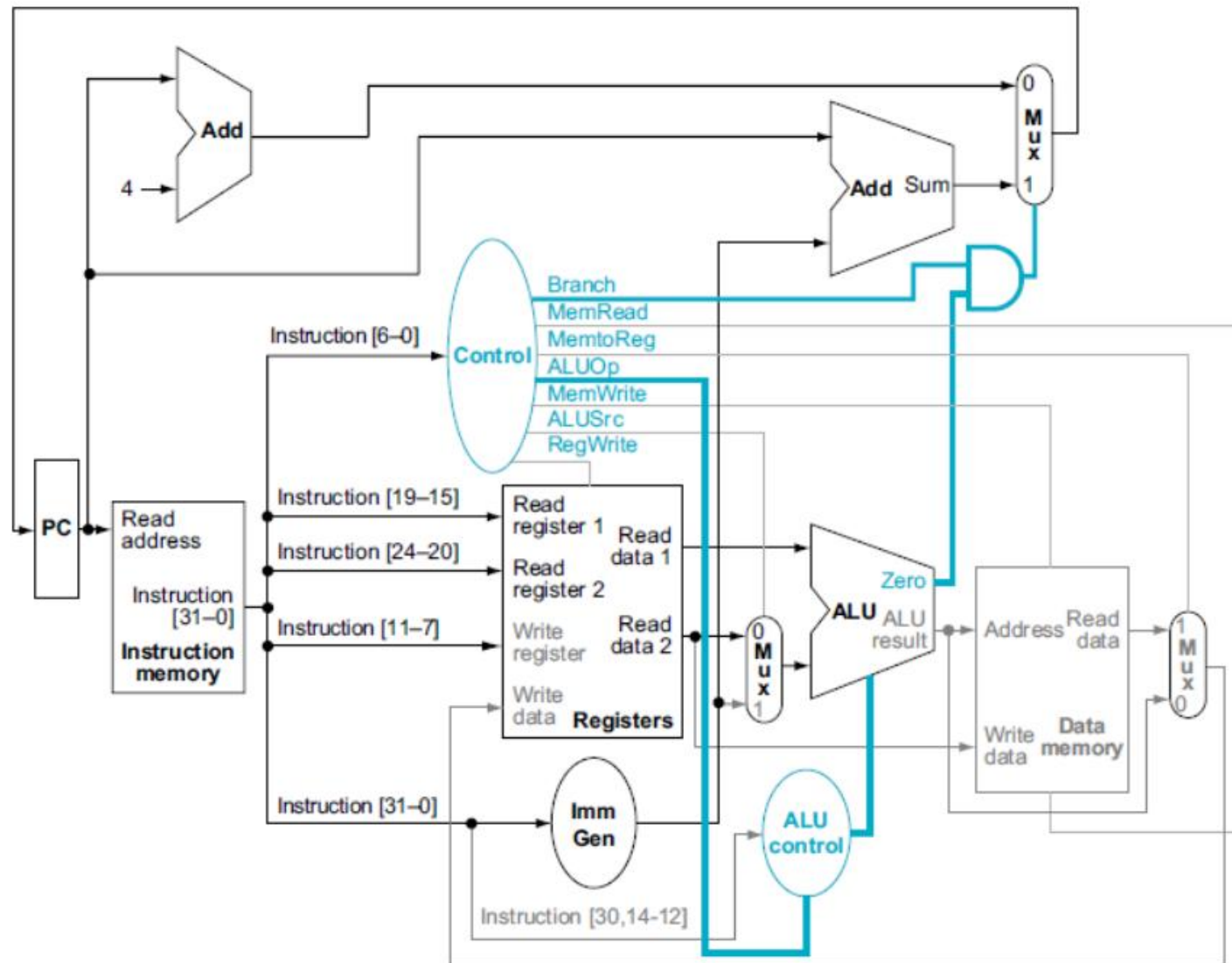


Load Instruction

- Highlighted control lines



Branch-on-Equal Instruction



Performance Issues

- Longest delay determines clock period
 - Critical path: load instruction
 - CPI ?
- Not feasible to vary period for different instructions
 - Violates design principle of *Making the common case fast*
- We will improve performance by pipelining

Chapter Outline

Chapter 4 – The Processor

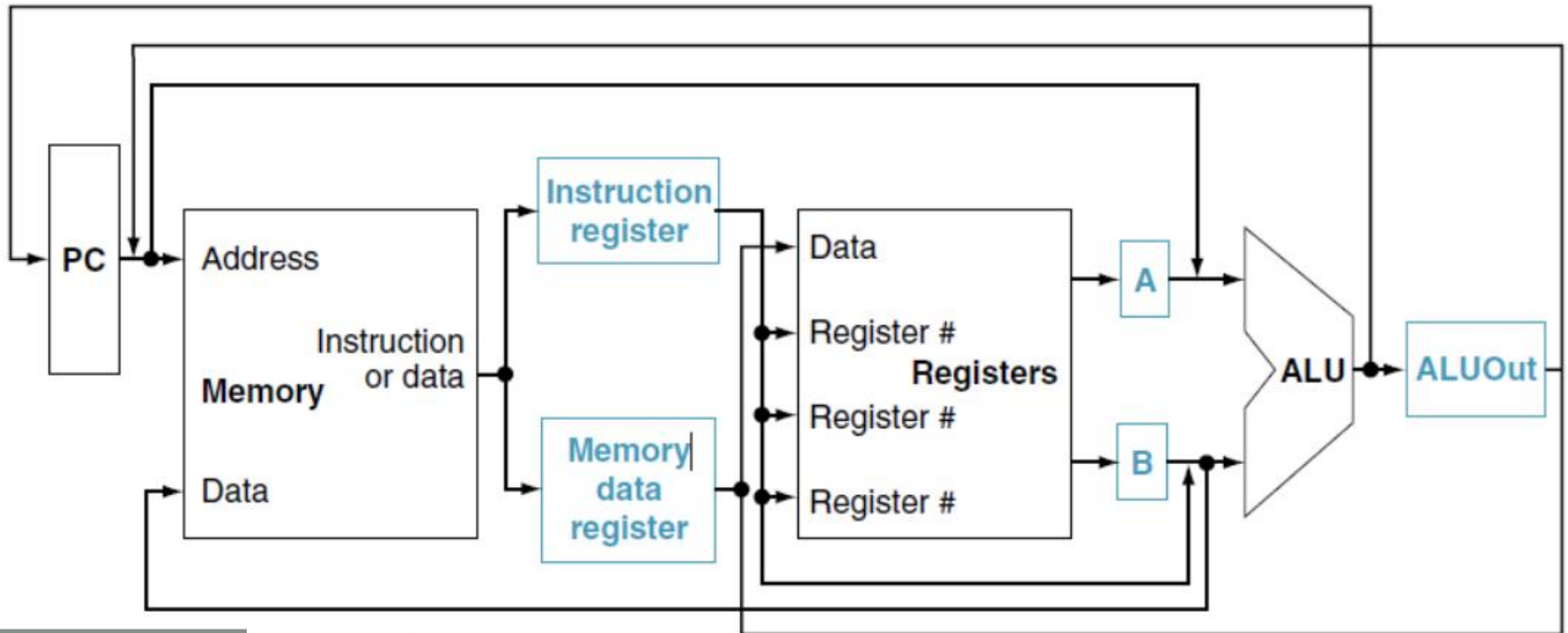
- 4.1 Introduction
- 4.2 Logic Design Conventions
- 4.3 Building a Datapath
- 4.4 A Simple Implementation Scheme
- 4.5 Multicycle Implementation
- 4.6 An Overview of Pipelining
- 4.7 Pipelined Datapath and Control
- 4.8 Data Hazards: Forwarding versus Stalling
- 4.9 Control Hazards
- 4.10 onwards (self-reading)

Introduction

- Single cycle implementation
- In a multicycle implementation, each *step* in the execution will take 1 clock cycle.
- Multicycle implementation has the following two advantages:
 - Different instructions to execute in different no. of clock cycles
 - Ability to share/re-use functional units within the execution of a single instruction

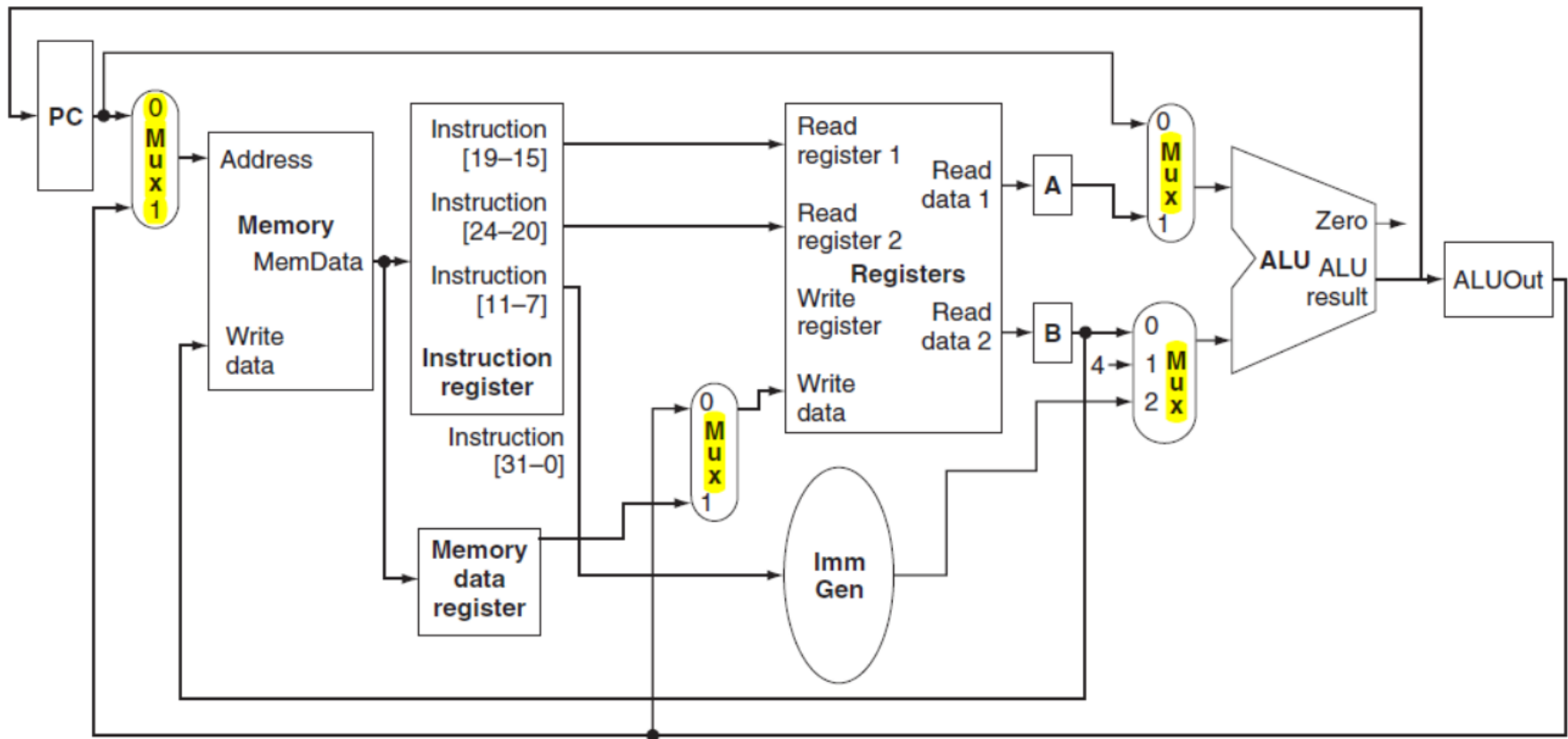
Multicycle datapath

- High-level view of the multicycle datapath
- Identify the differences with respect to the single-cycle datapath
- Steps executed in a single clock-cycle in this datapath
- Address of the single memory



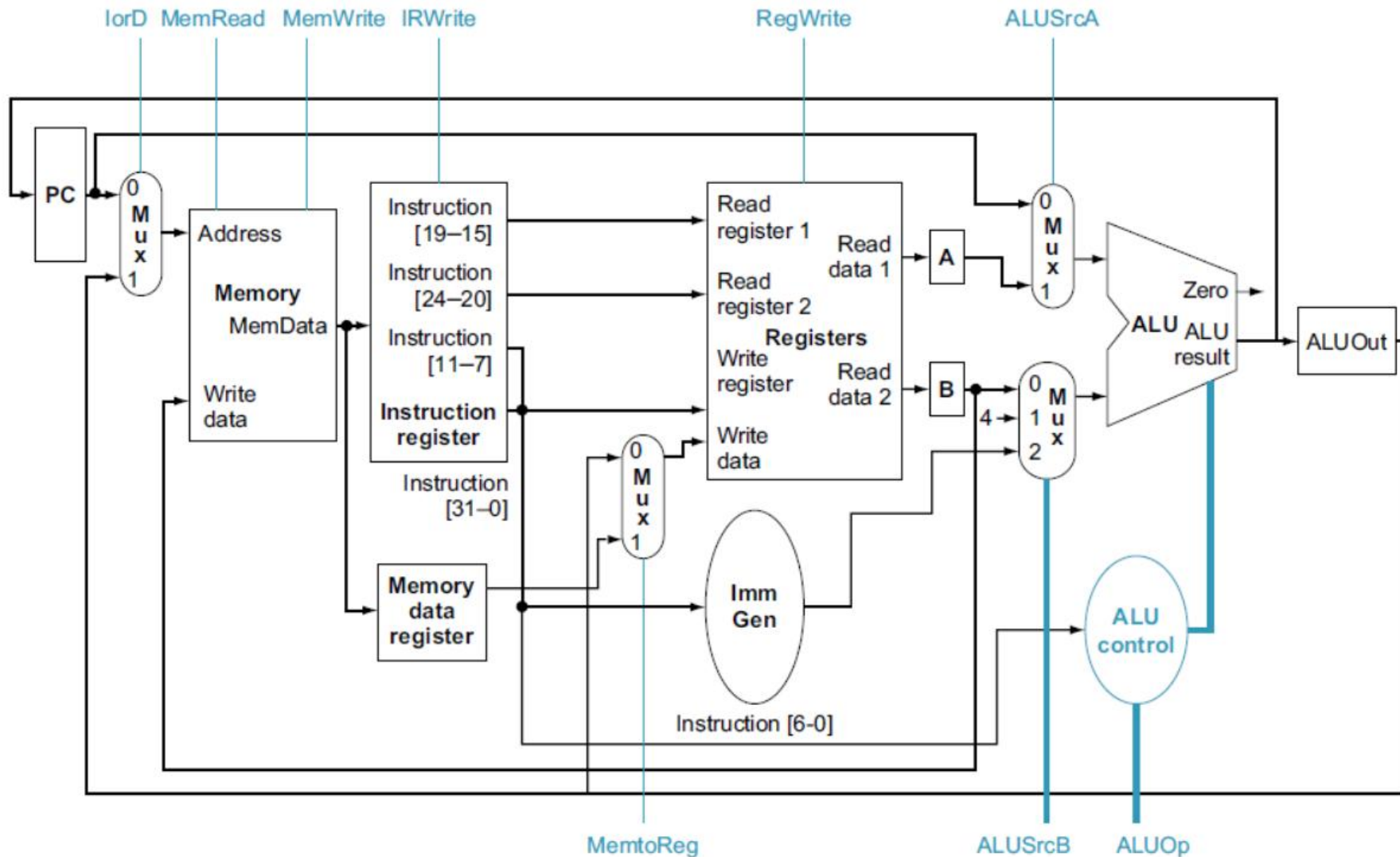
Multicycle datapath

- Multicycle datapath with multiplexers
- Size with respect to single-cycle implementation



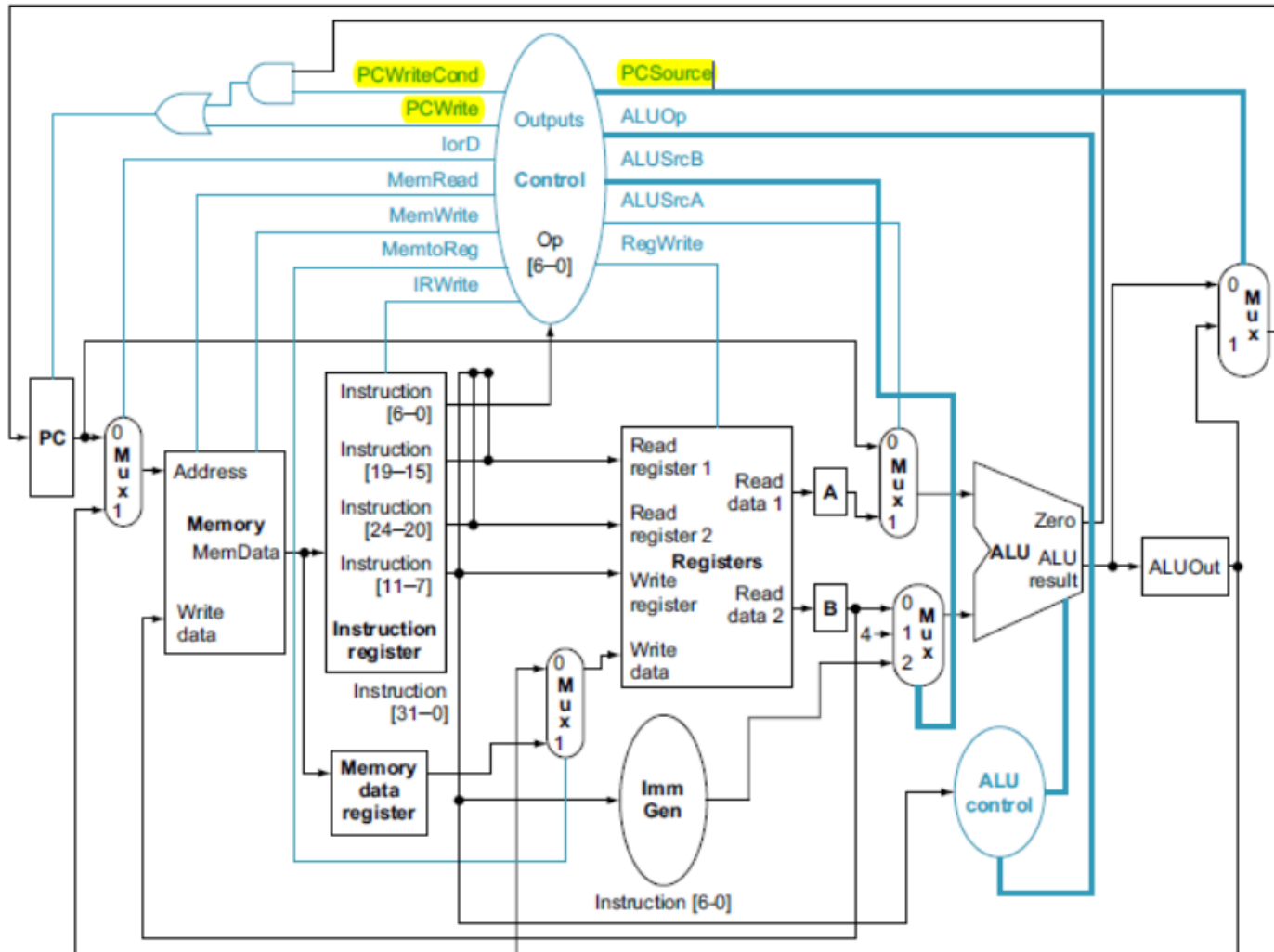
Multicycle datapath

- Multicycle datapath with control signals



Multicycle datapath

- Complete multicycle datapath including support for branch instructions



Multicycle datapath

Actions of the 1-bit control signals

Signal name	Effect when deasserted	Effect when asserted
RegWrite	None.	The general-purpose register selected by the Write register number is written with the value of the Write data input.
ALUSrcA	The first ALU operand is the PC.	The first ALU operand comes from the A register.
MemRead	None.	Content of memory at the location specified by the Address input is put on Memory data output.
MemWrite	None.	Memory contents at the location specified by the Address input is replaced by the value on the Write data input.
MemtoReg	The value fed to the register file Write data input comes from ALUOut.	The value fed to the register file Write data input comes from the MDR.
lorD	The PC is used to supply the address to the memory unit.	ALUOut is used to supply the address to the memory unit.
IRWrite	None.	The output of the memory is written into the IR.
PCWrite	None.	The PC is written; the source is controlled by PCSrc.
PCWriteCond	None.	The PC is written if the Zero output from the ALU is also active.

Actions of the 2-bit control signals

Signal name	Value (binary)	Effect
ALUOp	00	The ALU performs an add operation.
	01	The ALU performs a subtract operation.
	10	The funct field of the instruction determines the ALU operation.
ALUSrcB	00	The second input to the ALU comes from the B register.
	01	The second input to the ALU is the constant 4.
	10	The second input to the ALU is the immediate generated from the IR.
PCSrc	00	Output of the ALU ($PC + 4$) is sent to the PC for writing.
	01	The contents of ALUOut (the branch target address) are sent to the PC for writing.
	10	The jump target address ($IR[25:0]$ shifted left 2 bits and concatenated with $PC + 4[31:28]$) is sent to the PC for writing.

Multicycle datapath

Breaking the Instruction Execution into Clock Cycles

- Let us break instruction execution into a series of steps, where each step taken a single clock cycle
- This division is done such that major components (ALU, Memory, Register File) are only used once per step
- This also helps in keeping the clock cycle small

Multicycle datapath

Breaking the Instruction Execution into Clock Cycles

- Each RISC-V instruction needs from three to five of these steps:
 1. Step1: Instruction fetch step
 2. Step2: Instruction decode and register fetch step
 3. Step3: Execution, memory address computation, or branch completion
 4. Step4: Memory access or R-type instruction completion step
 5. Step5: Memory read completion step
- How many steps for R-type, memory-access, branch instructions?

Multicycle datapath

Step 1: Instruction fetch step

- Fetch instruction and load next instruction address in PC
- Generate appropriate control signals

Control Signal (1 bit)	Value
RegWrite	
ALUSrcA	0
MemRead	
MemWrite	
MemtoReg	
lorD	0
IRWrite	
PCWrite	1
PCWriteCond	

Control Signal (2 bit)	Value
ALUOp	00
ALUSrcB	01
PCSource	00

Multicycle datapath

Step 2: Instruction decode and register fetch step

- Act **optimistically** i.e. read registers and calc. branch address
- Generate appropriate control signals

Control Signal (1 bit)	Value
RegWrite	
ALUSrcA	0
MemRead	
MemWrite	
MemtoReg	
lorD	
IRWrite	
PCWrite	
PCWriteCond	

Control Signal (2 bit)	Value
ALUOp	00
ALUSrcB	10
PCSource	00